

Mobile Pentest

Name: Matyas Abreham

IDNO: CTC-207-26

Introduction and Approach

This mobile penetration-testing exercise focused on analyzing the InjuredAndroid application and locating the values required to complete the individual flag challenges. The investigation was mainly performed by reading the AndroidManifest.xml, examining decompiled Java code in JADX, tracing related classes and methods, and using ADB when an activity needed to be launched directly.

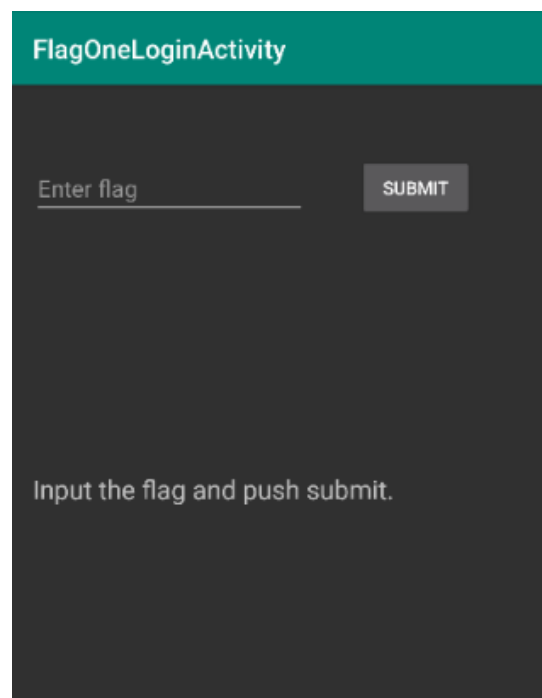
The challenges used several different techniques, including activity mapping, exported activities, hard-coded strings, Base64 decoding, Android broadcast receivers, DES-based encryption, SQLite-related code, and Firebase database access. For each flag, the approach was to follow the application's own logic and identify where the expected value was generated, stored, or retrieved.

Flag 1

Analysis: The first step was to identify where the application defines the activity responsible for the first challenge. AndroidManifest.xml is useful for this because it lists the application's activities and provides a quick way to map the challenge names to their corresponding classes.

After locating FlagOneLoginActivity in JADX, the submitFlag method was inspected. The comparison logic revealed the hard-coded value used by the application. This avoided guessing the password and showed exactly where the application checks the submitted flag.

The task is to find the login flag



Open injuredAndroid.apk in JADX go to Resources/res/ AndroidManifest.xml and search by clicking “ctrl f” then type flag_1 on the search bar

```
124 y
    'oid:theme="@style/AppTheme.NoActionBar"
    'oid:label="@string/title_activity_flag_ten_unicode"
    'oid:name="b3nac.injuredandroid.FlagTenUnicodeActivity"/>

128 y
    'oid:theme="@style/AppTheme.NoActionBar"
    'oid:label="@string/title_activity_flag_one_login"
    'oid:name="b3nac.injuredandroid.FlagOneLoginActivity"/>

132 y
    'oid:theme="@style/AppTheme.NoActionBar"
    'oid:label="@string/title_activity_flag_nine_firebase"
    'oid:name="b3nac.injuredandroid.FlagNineFirebaseActivity"/>

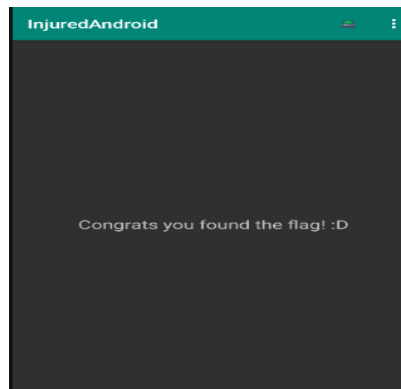
136 y
```

Then double click on `b3nac.injuredandroid.FlagOneLoginActivity`

```
@Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.AppCompatActivity, androidx.activity.ComponentActivity
protected void onCreate(Bundle bundle) {
    super.onCreate(bundle);
    setContentView(R.layout.activity_flag_one_login);
    j.j.a(this);
    C((Toolbar) findViewById(R.id.toolbar));
    ((FloatingActionButton) findViewById(R.id.fab)).setOnClickListener(new a());
}

public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText2);
    d.s.d.g.d(editText, "editText2");
    if (d.s.d.g.a(editText.getText().toString(), "Flag_One3")) {
        Intent intent = new Intent(this, (Class<?>) FlagOneSuccess.class);
        new FlagsOverview().J(true);
        new j().b(this, "flagOneButtonColor", true);
        startActivity(intent);
    }
}
```

Look at the submitFlag part there is the flag "F1ag_0n3"

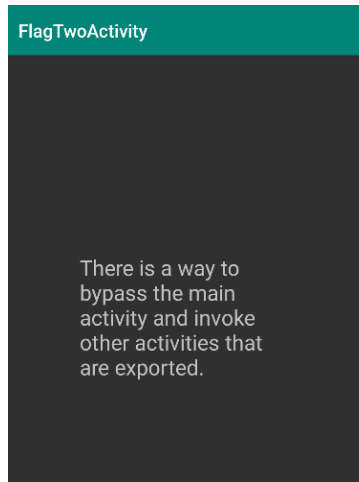


Flag 2

Analysis: This challenge did not provide a normal input field, so the activity itself had to be investigated. The exported="true" entries in AndroidManifest.xml were examined to identify activities that could be launched externally.

The activity chain eventually led to `b3nac.injuredandroid.b25IActivity`. Because the activity was exported, ADB could be used to start it directly with an explicit component name. The successful launch confirmed that the correct activity had been identified and opened the hidden challenge page.

Target: blank screen with no input file



go to `Resources/res/AndroidManifest.xml` and search by clicking “ctrl f” then search for events are marked with `exported="true"` then we found hit for an activity associated to `b3nac.injuredandroid.ExportedProtectedIntent`

```
FlagOneLoginActivity  FlagTwoActivity  AndroidManifest.xml  QXV0aA  ExportedProtectedIntent
1 package b3nac.injuredandroid;
2
3 import android.content.Intent;
4 import android.os.Bundle;
5 import android.view.View;
6 import androidx.appcompat.app.AppCompatActivity;
7 import androidx.appcompat.widget.Toolbar;
8 import com.google.android.material.floatingactionbutton.FloatingActionButton;
9 import com.google.android.material.snackbar.Snackbar;
10
11 /* Loaded from: classes.dex */
12 public class ExportedProtectedIntent extends AppCompatActivity {
13     private void f(Intent intent) {
14         Intent intent2 = (Intent) intent.getParcelableExtra("access_protected_component");
15         if (intent2.resolveActivity(getPackageManager()).getPackageName().equals("b3nac.injuredandroid")) {
16             startActivity(intent2);
17         }
18     }
19
20     /* JADX INFO: Access modifiers changed from: package-private */
21     public static /* synthetic */ void G(View view) {
22         Snackbar X = Snackbar.X(view, "Replace with your own action", 0);
23         X.Y("Action", null);
24         X.N();
25     }
26
27     /* JADX INFO: Access modifiers changed from: protected */
28     @Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.Fragment, androidx.activity.ComponentActivity, androidx.core.app.Activity, android.app.Activity
29     public void onCreate(Bundle bundle) {
30         super.onCreate(bundle);
31         setContentView(R.layout.activity_exported_protected_intent);
32         C((Toolbar) findViewById(R.id.toolbar));
33         ((FloatingActionButton) findViewById(R.id.fab)).setOnClickListener(a.f);
34     }
35
36     /* JADX INFO: Access modifiers changed from: protected */
37     @Override // androidx.fragment.app.Fragment, android.app.Activity
38     public void onResume() {
39         super.onResume();
40         f(getIntent());
41     }
42 }
```

Nothing is found. The next exported activity we find linked to the enigmatic class is `b3nac.injuredandroid.QXV0aA`

```

1 package b3nac.injuredandroid;
2
3 import android.os.Bundle;
4 import androidx.appcompat.app.AppCompatActivity;
5
6 /* Loaded from: classes.dex */
7 public final class b25lActivity extends AppCompatActivity {
8     /* JADX INFO: Access modifiers changed from: protected */
9     @Override // androidx.appcompat.app.AppCompatActivity, androidx.fragment.app.Fragment, androidx.activity.ComponentActivity, androidx.core.app.Activity, android.app.Activity
10     public void onCreate(Bundle bundle) {
11         super.onCreate(bundle);
12         setContentView(R.layout.activity_b25l);
13         j.j.a(this);
14         new FlagsOverview().M(true);
15         new j().b(this, "FlagTwoButtonColor", true);
16     }
17 }

```

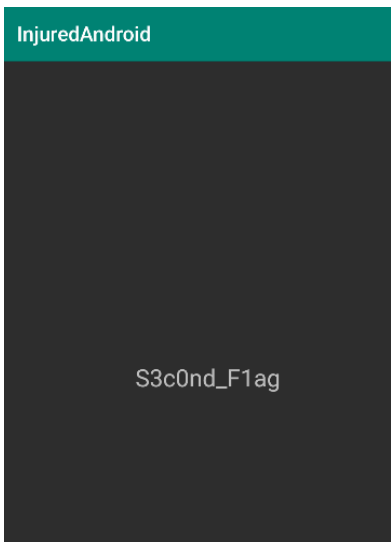
then there is activity linked to b3nac.injuredandroid.b25lActivity, so we found the key to unlock the page in the onCreate event. Let's call exported activities from ADB by using

"adb shell am start -n b3nac.injuredandroid/.b25lActivity"

```

$ adb shell am start -n b3nac.injuredandroid/.b25lActivity
Starting: Intent { cmp=b3nac.injuredandroid/.b25lActivity }

```

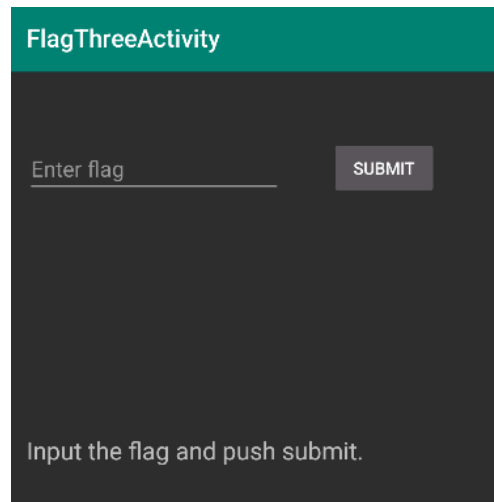


We successfully called the activity

Flag 3

Analysis: FlagThreeActivity was inspected directly in the decompiled source. A hard-coded string, "cmVzb3VyY2VzX3Lv", was found in the activity.

The string was then searched through the source to determine how it was used. The challenge was completed after identifying the value associated with that hard-coded data.

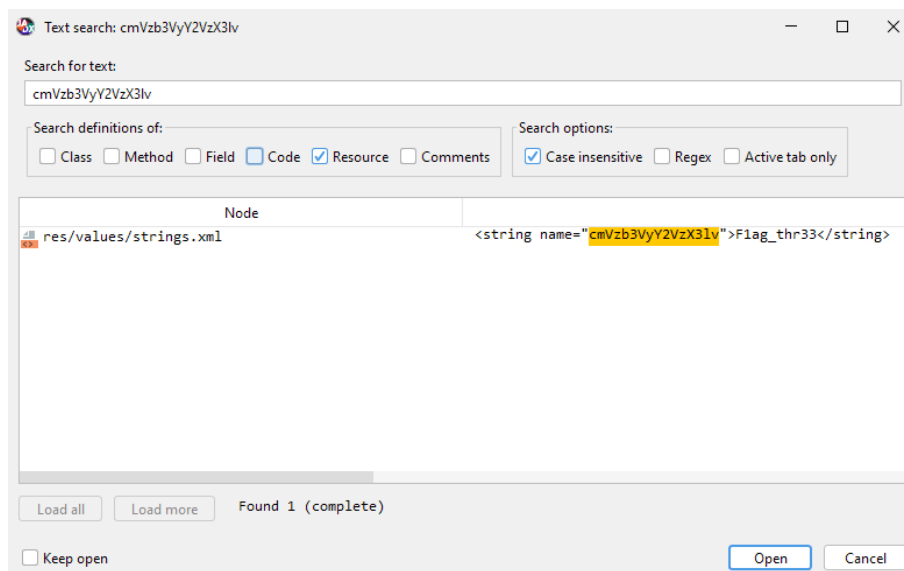


Go to Source/ b3nac.injuredandroid/ FlagThreeActivity

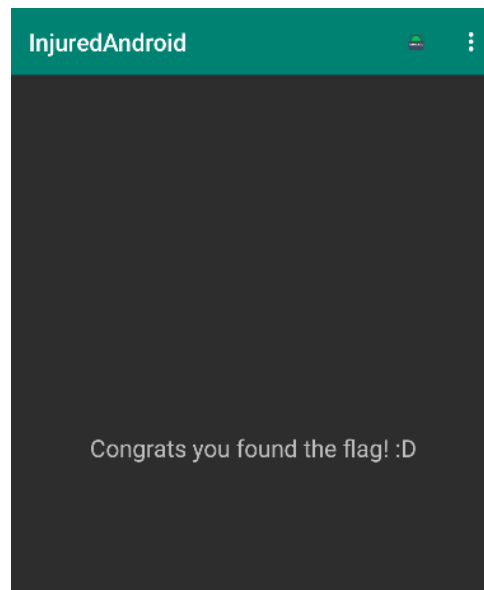
```
54
55     public final void submitFlag(View view) {
56         EditText editText = (EditText) findViewById(R.id.editText2);
57         d.s.d.g.d(editText, "editText2");
58         if (d.s.d.g.a(editText.getText().toString(), getString(R.string.cmVzb3VyY2VzX3lv))) {
59             Intent intent = new Intent(this, (Class<?>) FlagOneSuccess.class);
60             new FlagsOverview().L(true);
61             new j().b(this, "flagThreeButtonColor", true);
62             startActivity(intent);
63         }
64     }
65 }
```

There is a hard-coded string called “cmVzb3VyY2VzX3Lv”

Search for cmVzb3VyY2VzX3Lv



There is flag three. The task is completed.



Flag 4

Analysis: The important part of this challenge was understanding the expression `new String(a2, d.w.c.f2418a)`. The byte array `a2` contains the data that will become text, while `f2418a` specifies the character encoding used to interpret those bytes.

Tracing `d.w.c` showed that `f2418a` represents UTF-8. The next step was to trace how `a2` was created. The code `byte[] a2 = new g().a()` led to class `g`, where the encoded value was found.

The value `"NF9vdmVyZG9uZV9vbWVsZXRz"` was Base64-encoded. Decoding it produced `"4_overdone_omelets"`, which was then used as the flag.

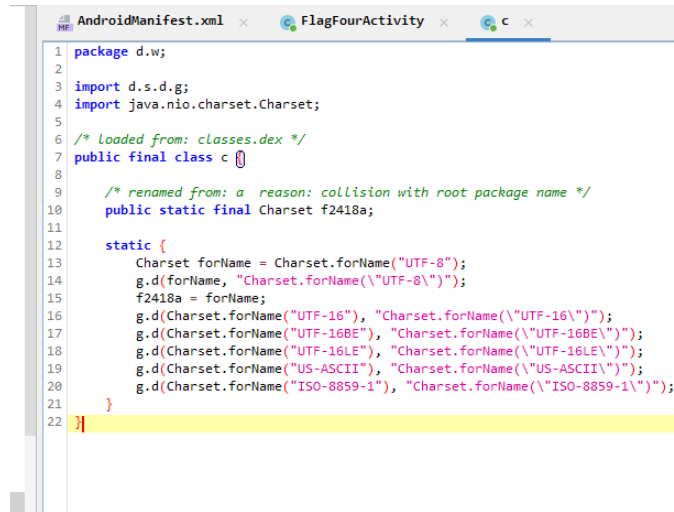
Go to Source/ b3nac.injuredandroid/ FlagFourActivity

```
public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText2);
    d.s.d.g.d(editText, "editText2");
    String string = editText.getText().toString();
    byte[] bArrA = new g().a();
    d.s.d.g.d(bArrA, "decoder.getData()");
    if (d.s.d.g.a(string, new String(bArrA, d.w.c.f2418a))) {
        Intent intent = new Intent(this, (Class<?>) FlagOneSuccess.class);
        new FlagsOverview().I(true);
        new j().b(this, "flagFourButtonColor", true);
        startActivity(intent);
    }
}
```

The app compares your input with `new String(a2, d.w.c.f2418a)`. this `d.w.c.f2418a` import must come from another base-level class. We can see the list of these classes in the sidebar

The first argument, a2, is a byte array. The second argument, d.w.c.f2418a, determines how those bytes are converted into text.

Searching the imported/base-level classes reveals d.w.c



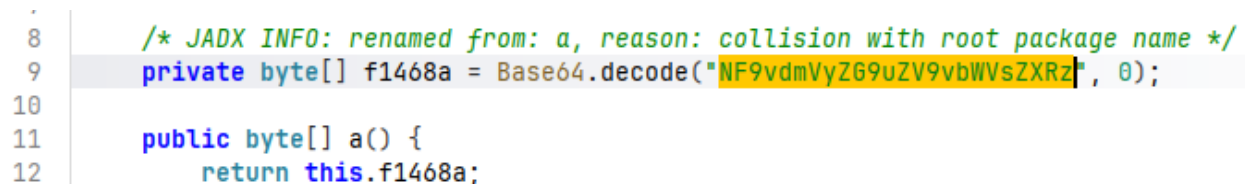
```
1 package d.w.c;
2
3 import d.s.d.g;
4 import java.nio.charset.Charset;
5
6 /* Loaded from: classes.dex */
7 public final class c {
8
9     /* renamed from: a reason: collision with root package name */
10    public static final Charset f2418a;
11
12    static {
13        Charset forName = Charset.forName("UTF-8");
14        g.d(forName, "Charset.forName(\"UTF-8\")");
15        f2418a = forName;
16        g.d(Charset.forName("UTF-16"), "Charset.forName(\"UTF-16\")");
17        g.d(Charset.forName("UTF-16BE"), "Charset.forName(\"UTF-16BE\")");
18        g.d(Charset.forName("UTF-16LE"), "Charset.forName(\"UTF-16LE\")");
19        g.d(Charset.forName("US-ASCII"), "Charset.forName(\"US-ASCII\")");
20        g.d(Charset.forName("ISO-8859-1"), "Charset.forName(\"ISO-8859-1\")");
21    }
22 }
```

Inside this class, f2418a represents the **UTF-8 charset**.

Looking up a few lines in FlagFourActivity, we see that a2 is a byte array defined as

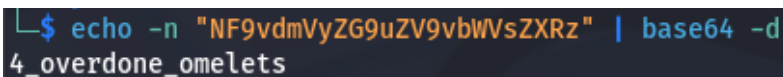
```
byte[] a2 = new g().a()
```

our first thought is to look at the other classes defined in b3nac.injuredandroid. go g class



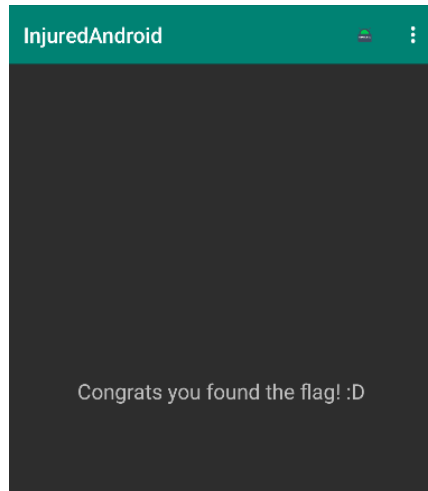
```
8 /* JADX INFO: renamed from: a, reason: collision with root package name */
9 private byte[] f1468a = Base64.decode("NF9vdmVyZG9uZV9vbWVsZXRz", 0);
10
11 public byte[] a() {
12     return this.f1468a;
13 }
```

We found the Base64-encoded value "NF9vdmVyZG9uZV9vbWVsZXRz" let's decode it



```
$ echo -n "NF9vdmVyZG9uZV9vbWVsZXRz" | base64 -d
4_overdone_omelets
```

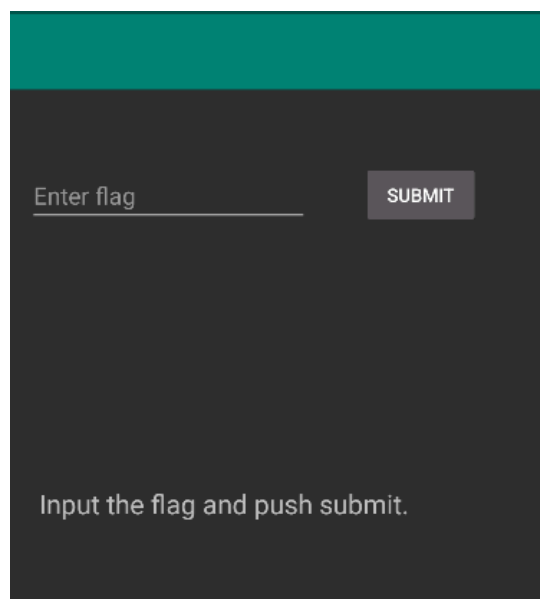
Put the decoded flag "4_overdone_omelets"



Flag 5

Analysis: This challenge depends on an Android BroadcastReceiver rather than a simple hard-coded value. FlagFiveActivity sends the custom intent `com.b3nac.injuredandroid.intent.action.CUSTOM_INTENT`, so the receiver handling that intent was the next part of the investigation.

Inside FlagFiveReceiver, the variable `f1454a` was used as a counter. The receiver increases this value each time `onReceive()` executes, and the flag becomes unlocked when the value reaches 2. Opening the challenge again causes the broadcast to be sent again, allowing the receiver to execute a second time.



Go to `Resources/res/` AndroidManifest.xml and search for `flag_five`

```

AndroidManifest.xml
Find: flag_five
106 </intent-filter>
107 </activity>
108 <activity android:name="b3nac.injuredandroid.SettingsActivity"/>
109 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_exported_protected_intent" android:name="b3nac.injuredandroid.Expo
110 <activity android:name="b3nac.injuredandroid.QXV0aA" android:exported="true"/>
111 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_twelve_protected" android:name="b3nac.injuredandroid.FlagTwel
112 <activity android:label="@string/title_activity_deep_link" android:name="b3nac.injuredandroid.DeepLinkActivity">
113 <intent-filter android:label="filter_view_flag11">
114 <action android:name="android.intent.action.VIEW"/>
115 <category android:name="android.intent.category.DEFAULT"/>
116 <category android:name="android.intent.category.BROWSABLE"/>
117 <data android:scheme="flag11"/>
118 </intent-filter>
119 <intent-filter android:label="filter_view_flag11">
120 <action android:name="android.intent.action.VIEW"/>
121 <category android:name="android.intent.category.DEFAULT"/>
122 <category android:name="android.intent.category.BROWSABLE"/>
123 <data android:scheme="https"/>
124 </intent-filter>
125 </activity>
126 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_ten_unicode" android:name="b3nac.injuredandroid.FlagTenUnicod
127 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_one_login" android:name="b3nac.injuredandroid.FlagOneLoginAct
128 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_nine_firebase" android:name="b3nac.injuredandroid.FlagNineFir
129 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_eight_login" android:name="b3nac.injuredandroid.FlagEightLogi
130 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_seven_sqlite" android:name="b3nac.injuredandroid.FlagsSevenSql
131 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flags_overview" android:name="b3nac.injuredandroid.FlagsOverview"
132 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_six_login" android:name="b3nac.injuredandroid.FlagSixLoginAct
133 </activity>
134 <activity android:name="b3nac.injuredandroid.MainActivity">
135 <intent-filter>
136 <action android:name="android.intent.action.MAIN"/>
137 <category android:name="android.intent.category.LAUNCHER"/>
138 </intent-filter>
139 </activity>
140 <activity android:name="b3nac.injuredandroid.XSSTextActivity"/>
141 <activity android:name="b3nac.injuredandroid.DisplayPostXSS"/>
142 <activity android:name="b3nac.injuredandroid.FlagOneSuccess"/>
143 <activity android:name="b3nac.injuredandroid.b251Activity" android:exported="true"/>
144 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_two" android:name="b3nac.injuredandroid.FlagTwoActivity"/>
145 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_three" android:name="b3nac.injuredandroid.FlagThreeActivity"/
146 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_four" android:name="b3nac.injuredandroid.FlagFourActivity"/>
147 <receiver android:name="b3nac.injuredandroid.FlagFiveReceiver" android:exported="true"/>
148 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_five" android:name="b3nac.injuredandroid.FlagFiveActivity"/>

```

Open b3nac.injuredandroid.FlagFiveActivity

```

46 public void onCreate(Bundle bundle) {
47     super.onCreate(bundle);
48     setContentView(R.layout.activity_flag_five);
49     C((Toolbar) findViewById(R.id.toolbar));
50     ((FloatingActionButton) findViewById(R.id.fab)).setOnClickListener(new View.OnClickListener() { // from class: b3nac.injuredandroid.c
51         @Override // android.view.View.OnClickListener
52         public final void onClick(View view) {
53             FlagFiveActivity.this.G(view);
54         }
55     });
56     new ComponentName(this, FlagFiveReceiver.class);
57     getPackageManager();
58     a.b(this).c(this.x, new IntentFilter("com.b3nac.injuredandroid.intent.action.CUSTOM_INTENT"));
59     ((Button) findViewById(R.id.button9)).setOnClickListener(new View.OnClickListener() { // from class: b3nac.injuredandroid.b
60         @Override // android.view.View.OnClickListener
61         public final void onClick(View view) {
62             FlagFiveActivity.this.H(view);
63         }
64     });
65 }

```

Looking at onCreate(), we find code that eventually calls H() then F())

```

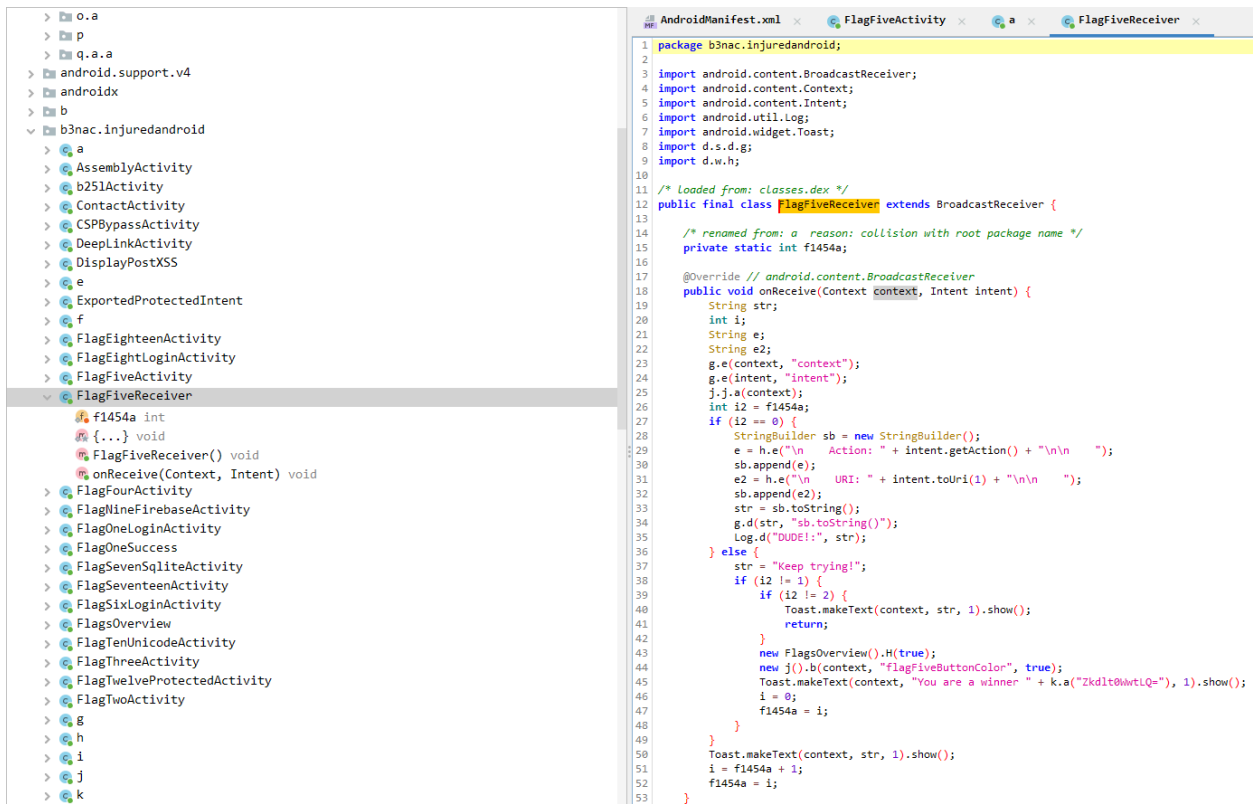
public void F() {
    sendBroadcast(new Intent("com.b3nac.injuredandroid.intent.action.CUSTOM_INTENT"));
}

```

It sends a Broadcast using this custom Intent

“com.b3nac.injuredandroid.intent.action.CUSTOM_INTENT”.

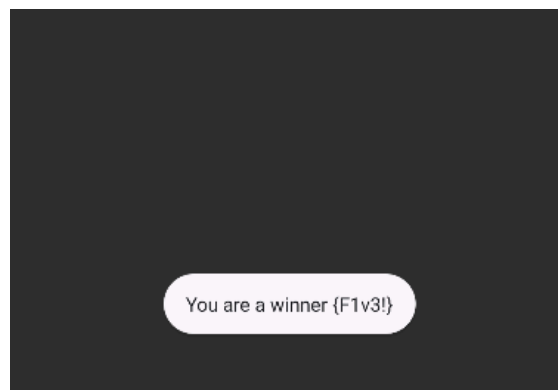
The obvious next step is to examine the decompiled FlagFiveReceiver



Inside FlagFiveReceiver, we find “ f1454a”, so the flag is unlocked when f1454a == 2

The important part is that f1454a increases by **1 every time onReceive() runs**.

Simply opening the challenge again sends the Broadcast again, causing the receiver to run a second time.



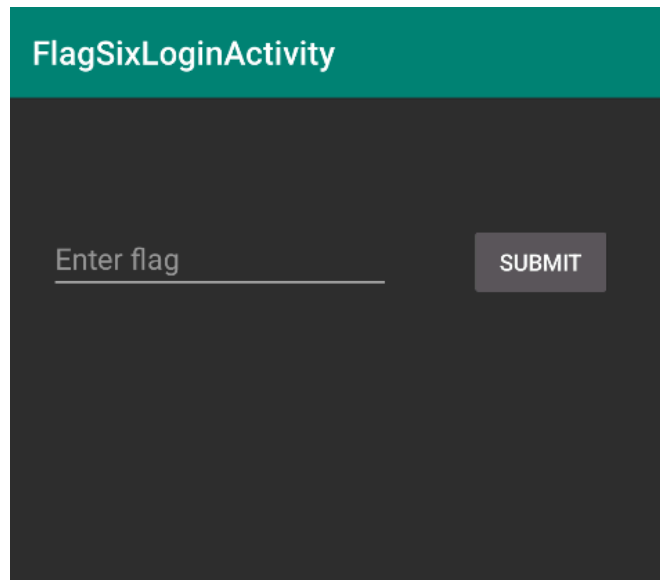
Flag 6

Analysis: FlagSixLoginActivity compares the user's input against a value produced by k.a(). Tracing the k class showed that the challenge uses DES encryption, so the next step was to follow how the DES key is generated.

The code uses SecretKeyFactory.getInstance("DES") together with DESKeySpec(f1472a). The variable f1472a comes from h.b(), which in turn uses the

encoded value "Q2FwdHVyM1RoMXM". This showed that the key material and encrypted data had to be traced through the related helper classes rather than guessed.

Python was then used as a decoding aid. The resulting value was "This_Isn't_Where_I_Parked_My_Car", which was used as the flag.



Go to Resources/res/ AndroidManifest.xml and search for flag_six

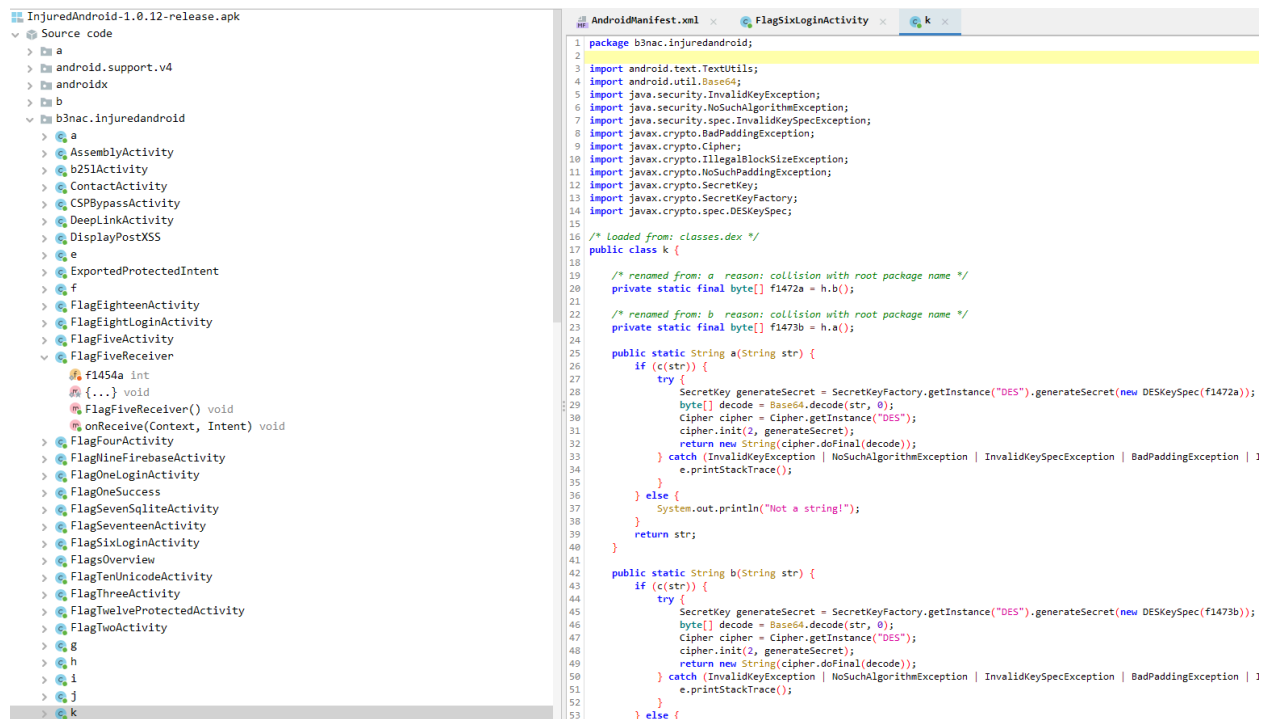
```
Find: flag_six
80 </intent-filter>
76 </activity>
91 <activity android:name="b3nac.injuredandroid.SettingsActivity"/>
92 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_exported_protected_intent" android:name="b3nac.injuredandroid.ExportedProtec
97 <activity android:name="b3nac.injuredandroid.QXV0aA" android:exported="true"/>
100 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_twelve_protected" android:name="b3nac.injuredandroid.FlagTwelveProtecte
104 <activity android:label="@string/title_activity_deep_link" android:name="b3nac.injuredandroid.DeepLinkActivity">
107 <intent-filter android:label="filter_view_flag11">
108 <action android:name="android.intent.action.VIEW"/>
110 <category android:name="android.intent.category.DEFAULT"/>
111 <category android:name="android.intent.category.BROWSABLE"/>
113 <data android:scheme="flag11"/>
107 </intent-filter>
115 <intent-filter android:label="filter_view_flag11">
116 <action android:name="android.intent.action.VIEW"/>
118 <category android:name="android.intent.category.DEFAULT"/>
119 <category android:name="android.intent.category.BROWSABLE"/>
121 <data android:scheme="https"/>
115 </intent-filter>
104 </activity>
124 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_ten_unicode" android:name="b3nac.injuredandroid.FlagTenUnicodeActivity"/>
128 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_one_login" android:name="b3nac.injuredandroid.FlagOneLoginActivity"/>
132 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_nine_firebase" android:name="b3nac.injuredandroid.FlagNineFirebaseActiv
136 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_eight_login" android:name="b3nac.injuredandroid.FlagEightLoginActivity"/>
140 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_seven_sqlite" android:name="b3nac.injuredandroid.FlagSevenSqliteActivit
144 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flags_overview" android:name="b3nac.injuredandroid.FlagsOverview"/>
148 <activity android:theme="@style/AppTheme.NoActionBar" android:label="@string/title_activity_flag_six_login" android:name="b3nac.injuredandroid.FlagSixLoginActivity"/>
```

Open b3nac.injuredandroid.FlagSixLoginActivity

```
public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText3);
    g.d(editText, "editText3");
    if (g.a(editText.getText().toString(), k.a("k3FE1EG9lnowB0ateGhj5px6QsXRNJKh//8Jxi8KXW7iDpk2xRxhQ=="))) {
        Intent intent = new Intent(this, FlagOneSuccess.class);
        FlagsOverview.g = true;
        new j().b(this, "flagSixButtonColor", true);
        startActivity(intent);
    }
}
```

The application compares our input with a value generated by calling k.a() on what appears to be a Base64-encoded string

we can find the k class inside b3nac.injuredandroid



the application uses **DES** for encryption. The decryption code contains
`SecretKey generateSecret = SecretKeyFactory.getInstance("DES").generateSecret(new DESKeySpec(f1472a));`

Looking at the class definition `f1472a = h.b()`, so the function `h.b()` returns a value based on "Q2FwdHVyM1RoMXM"

We need a decoder so we use Python code to decode

```
import base64
from Crypto.Cipher import DES
from Crypto.Util.Padding import unpad

secret = "Q2FwdHVyM1RoMXM="
stringtodecode = "k3FE1EG9lnoWbOateGhj5pX6QsXRNJKh///8Jxi8KXW7iDpk2xRxhQ=="

def decrypt():
    secret_byte_array = base64.b64decode(secret)[:8]
    string_byte_array = base64.b64decode(stringtodecode)

    deshandler = DES.new(secret_byte_array, DES.MODE_ECB)

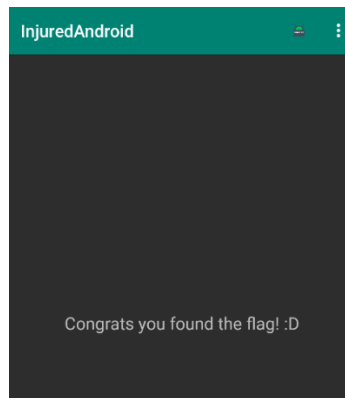
    decrypted_bytes = deshandler.decrypt(string_byte_array)
    clean_text = unpad(decrypted_bytes, DES.block_size).decode('utf-8')

    print(f"Decrypted Flag: {clean_text}")

decrypt()
```

```
└─$ python3 decoder.py  
Decrypted Flag: {This_Isn't_Where_I_Parked_My_Car}
```

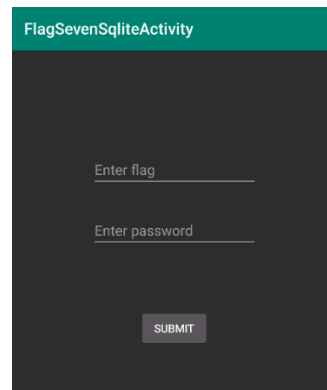
The decoder gives as {This_Isn't_Where_I_Parked_My_Car} put it as a flag



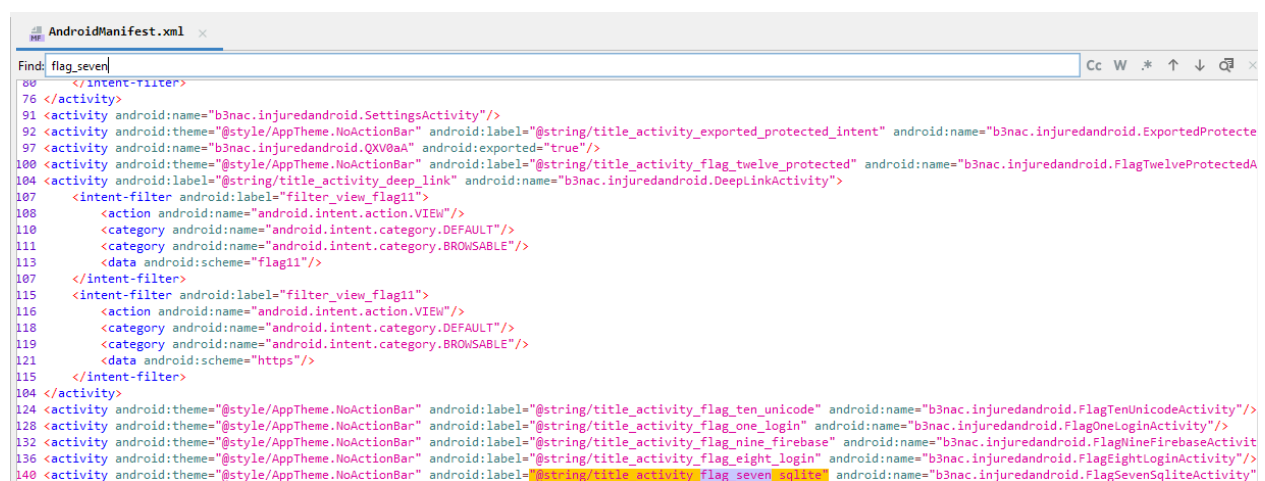
Flag 7

Analysis: FlagSevenActivity was inspected first, followed by FlagSevenSqliteActivity. The application compares the submitted input with a value obtained through `b3nac.injuredandroid.j`, indicating that the expected value is retrieved through another part of the application rather than being written directly in the activity.

This screenshot set documents the source-code path up to the comparison point in `submitFlag`, where the input is checked against a value retrieved via `b3nac.injuredandroid.j` rather than a literal hard-coded string. The class(es) that generate or store that value were not captured in this session, so the final flag string is left as a follow-up item pending re-verification against the device/emulator rather than reported from memory.



Go to Source/ b3nac.injuredandroid/ FlagSevenActivity



Double click on b3nac.injuredandroid.FlagSevenSqliteActivit

```

public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText8);
    d.s.d.g.d(editText, "editText8");
    String string = editText.getText().toString();
    EditText editText2 = (EditText) findViewById(R.id.editText7);
    d.s.d.g.d(editText2, "editText7");
    String string2 = editText2.getText().toString();
    j jVar = new j();
    String strC = jVar.c("flagSevenEncrypted", "");
    String strC2 = jVar.c("flagSevenPasswordEncrypted", "");
    if (!d.s.d.g.a(string, strC) || !d.s.d.g.a(string2, strC2)) {
        Toast.makeText(this, "Try again! :D", 0).show();
        return;
    }
    FlagsOverview.H = true;
    j jVar2 = new j();
    Context applicationContext = getApplicationContext();
    d.s.d.g.d(applicationContext, "applicationContext");
    jVar2.b(applicationContext, "flagSevenButtonColor", true);
    F();
}
}

```

The application compares our input with value obtained from b3nac.injuredandroid.j

Flag 8

Analysis: In FlagEightLoginActivity, the submitted value is passed to this.y.b(). Following the y object showed that it is connected to a Firebase database and that the application accesses a child named aws.

The Firebase database URL was located in resources.arsc/rec/values/strings.xml by searching for firebase_database. The database endpoint was then examined with the aws child in JSON form. This demonstrates how a value stored remotely can be discovered by tracing the application's database configuration and the child key it requests.

Go to Resources/res/ AndroidManifest.xml and search for flag_eight

```
Find: Flag_eight
136      android:label="@string/title_activity_flag_nine_firebase
      android:name="b3nac.injuredandroid.FlagNineFirebaseActivity"/>
      <activity
      android:theme="@style/AppTheme.NoActionBar"
      android:label="@string/title_activity_flag_eight_login"
      android:name="b3nac.injuredandroid.FlagEightLoginActivity"/>
140    </activity>
```

Double click on b3nac.injuredandroid.FlagEightLoginActivity

```
129
130 public final void submitFlag(View view) {
131     EditText editText = (EditText) findViewById(R.id.editText9);
132     d.s.d.g.d(editText, "editText2");
133     this.y.b(new c(editText.getText().toString()));
134 }
```

The submitFlag passing it to a strange function this.y.b(). Let's check function y

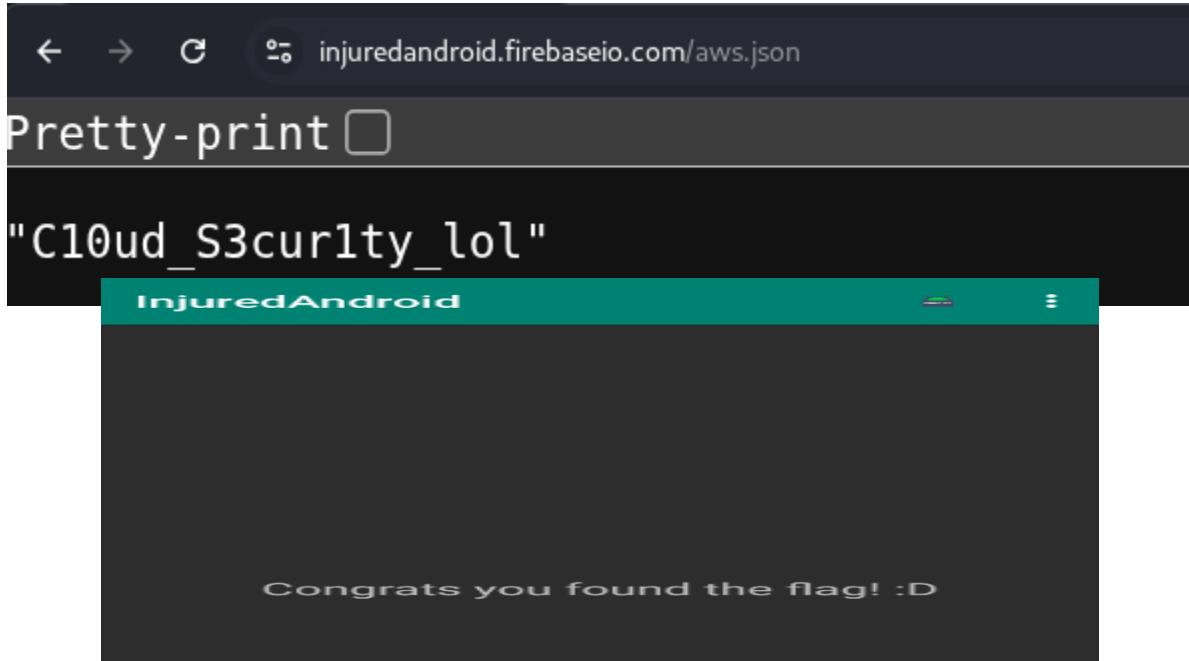
"this.y" is connected to a Firebase database and the application is accessing a firebase child "aws"

```
public FlagEightLoginActivity() {
    com.google.firebase.database.f fVarB = com.google.firebase.database.f.b();
    d.s.d.g.d(fVarB, "FirebaseDatabase.getInstance()");
    com.google.firebase.database.d dVarD = fVarB.d();
    d.s.d.g.d(dVarD, "FirebaseDatabase.getInstance().reference");
    this.x = dVarD;
    com.google.firebase.database.d dVarH = dVarD.h("/aws");
    d.s.d.g.d(dVarH, "database.child(\"/aws\")");
    this.y = dVarH;
}
```

Let's see the database connection, go to resources.arsc/rec/values/strings.xml and search for firebase_database


```
Find: firebase_database
78 <string name="error_icon_content_description">Error</string>
79 <string name="exposed_dropdown_menu_content_description">Show dropdown menu</string>
80 <string name="fab_transformation_scrim_behavior">com.google.android.material.transformation.FabTransformationScrimBehavior</string>
81 <string name="fab_transformation_sheet_behavior">com.google.android.material.transformation.FabTransformationSheetBehavior</string>
82 <string name="firebase_database_url">https://injuredandroid.firebaseio.com</string>
83
```

Copy the url ("https://injuredandroid.firebaseio.com") and add the child aws.json search it using browser. (url= <https://injuredandroid.firebaseio.com/aws.json>)



Flag 9

Analysis: FlagNineLoginActivity follows a process similar to Flag 8, but the database child name is encoded. The code was traced by cross-referencing B and following the first related occurrence to understand how the child name is generated.

After identifying the encoded child name, Base64 decoding was used to determine the required database path. The same Firebase endpoint was then used with the flags.json child to inspect the corresponding data.

Go to Resources/res/ AndroidManifest.xml and search for flag_nine

```
132 <activity
    android:name="b3nac.injuredandroid.FlagOneLoginActivity"/>
    android:theme="@style/AppTheme.NoActionBar"
    android:label="@string/title_activity_flag_nine_firebase"
    android:name="b3nac.injuredandroid.FlagNineFirebaseActivity"/>
136 <activity
```

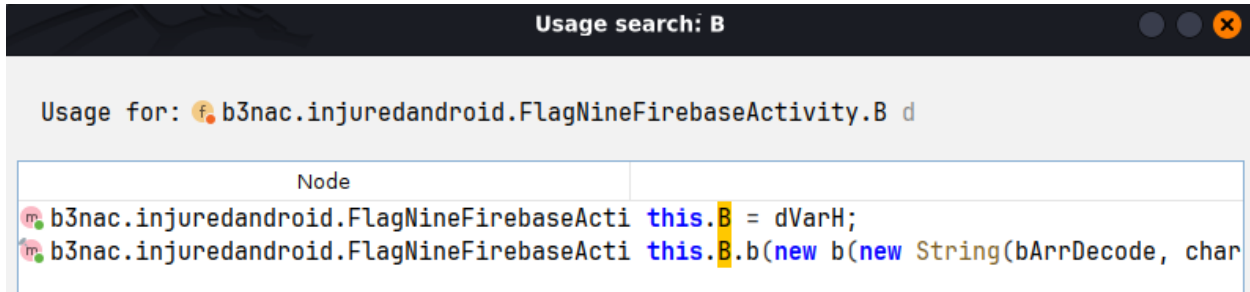
Double click on b3nac.injuredandroid.FlagNineLoginActivity

```

public final void submitFlag(View view) {
    EditText editText = (EditText) findViewById(R.id.editText2);
    d.s.d.g.d(editText, "editText2");
    byte[] bArrDecode = Base64.decode(editText.getText().toString(), 0);
    d.s.d.g.d(bArrDecode, "decodedPost");
    Charset charset = StandardCharsets.UTF_8;
    d.s.d.g.d(charset, "StandardCharsets.UTF_8");
    this.B.b(new b(new String(bArrDecode, charset)));
}

```

Put the cursor on B and Click x to cross reference it



Usage search: B

Usage for:  b3nac.injuredandroid.FlagNineFirebaseActivity.B d

Node	
 b3nac.injuredandroid.FlagNineFirebaseActi	this.B = dVarH;
 b3nac.injuredandroid.FlagNineFirebaseActi	this.B.b(new b(new String(bArrDecode, char

Double click on the first B

```

public FlagNineFirebaseActivity() {
    byte[] bArrDecode = Base64.decode("ZmxhZ3Mv", 0);
    this.y = bArrDecode;
    d.s.d.g.d(bArrDecode, "decodedDirectory");
    Charset charset = StandardCharsets.UTF_8;
    d.s.d.g.d(charset, "StandardCharsets.UTF_8");
    this.z = new String(bArrDecode, charset);
    com.google.firebase.database.f fVarB = com.google.firebase.database.f.b();
    d.s.d.g.d(fVarB, "FirebaseDatabase.getInstance()");
    com.google.firebase.database.d dVarD = fVarB.d();
    d.s.d.g.d(dVarD, "FirebaseDatabase.getInstance().reference");
    this.A = dVarD;
    com.google.firebase.database.d dVarH = dVarD.h(this.z);
    d.s.d.g.d(dVarH, "database.child(refDirectory)");
    this.B = dVarH;
}

```

The process is the same with flag 8 the only difference is child name is encoded with base64 so let's decode it

```

$ echo -n "ZmxhZ3Mv" | base64 -d
flags/

```

Use the same url and change the child to flags.json
<https://injuredandroid.firebaseio.com/flags.json>

